Models of computation

Last modified: 2026-11-11 01:02:38 (PST)

To say that a problem needs a certain number of steps, we must first fix what a step is. This page defines the Turing machine, the model of computation that complexity theory uses, and shows why the choice of model hardly matters: the variants of the model, and ordinary computers, simulate each other with at most a polynomial increase in running time. It ends with the universal machine, one Turing machine that can run every other. The asymptotic notation page defines the $O(\cdot)$ notation used here.

Turing machines

Definition 0.1 (Turing machine)**.** A **Turing machine** with $k \geq 2$ tapes consists of:

- k tapes, each a sequence of cells that starts at a leftmost cell and goes on forever to the right, with one head per tape that reads and writes one cell at a time. The first tape is the *input tape*, which the machine can read but not write; the last tape is the *output tape*; any others are *work tapes*.
- A finite *alphabet* A of the symbols a cell can hold. It includes the binary digits 0 and 1, a blank symbol $\square$, and a start symbol $\triangleright$.
- A finite set Q of *states*, which includes a start state q_{start} and a halting state q_{halt}.
- A *transition function*, a map from $Q \times A^k$ to $Q \times A^{k-1} \times \{\text{L}, \text{S}, \text{R}\}^k$.

The machine starts in state q_{start} with every head on its tape's leftmost cell. Each tape holds $\triangleright$ in its leftmost cell and blanks after it, except that the input tape holds the input $x \in \{0,1\}^*$ right after the $\triangleright$. In one *step*, the machine reads its state and the k symbols under its heads, and the transition function gives a new state, a symbol to write on each tape but the input tape, and a move for each head: one cell left (L), stay (S), or one cell right (R). A head told to move left from the leftmost cell stays where it is. The machine *halts* when it enters q_{halt}, and its *output* is then the string of binary digits on the output tape, read from the cell after $\triangleright$ up to the first symbol that is not 0 or 1 (Arora and Barak 2009, sec. 1.2).

Example 0.1 (A machine that checks for palindromes)**.** A binary string is a *palindrome* if it reads the same backward as forward, like 0110 or 10101. A Turing machine with three tapes (input, one work tape, and output) can check whether its input x is a palindrome in three phases (Arora and Barak 2009, example 1.1):

1. *Copy*: move the input head and the work head right together, writing each symbol of x onto the work tape, until the input head reaches the blank after x. The first step also moves the output head right, off $\triangleright$.
2. *Rewind*: move the input head left until it reads $\triangleright$, then one cell right, onto the first symbol of x. Move the work head one cell left, onto the last symbol of the copy.
3. *Compare*: move the input head right and the work head left together. At each step, first check the input head: if it reads the blank after x, write 1 on the output tape and halt. Otherwise, if the two heads read different symbols, write 0 on the output tape and halt.

On the input 0110, the compare phase reads the pairs $(0,0)$, $(1,1)$, $(1,1)$, and $(0,0)$, finds no mismatch, and outputs 1. On 0111 it reads $(0,1)$ first and outputs 0. Each phase is a few states of the machine, and each state of a phase checks which symbols the heads read to decide whether to stay in the phase or move to the next.

Definition 0.2 (Computing a function)**.** Let f be a function from $\{0,1\}^*$ to $\{0,1\}^*$. A Turing machine M **computes** f if M halts on every input $x \in \{0,1\}^*$ with output $f(x)$. Write $M(x)$ for the output of M on input x. A Turing machine decides a decision problem L exactly when it computes the function that is 1 on L and 0 off it (Arora and Barak 2009, sec. 1.2).

Example 0.2 (Palindromes in linear time)**.** Let PAL be the decision problem whose yes inputs are the palindromes. The machine of Example 0.1 computes the function that is 1 on palindromes and 0 on other strings, so it decides PAL. On an input of length n, the copy phase takes $n+1$ steps, the rewind phase at most $n+2$, and the compare phase at most $n+1$, so it runs in time $3n+4$, which is $O(n)$, and PAL is in P.

Remark 0.1 (Steps and running time). The running time of a Turing machine counts the steps of Definition 0.1, one application of the transition function each. One step moves each head by at most one cell, so reaching a cell m cells away takes at least m steps. An ordinary computer can jump to any memory address in one operation, so a Turing machine can need more steps than an ordinary program for the same task. Theorem 0.1 and Remark 0.4 explain why that gap does not change which problems have polynomial-time algorithms.

Robustness of the model

Definition 0.3 (Time-constructible function)**.** A function T from the nonnegative integers to the nonnegative integers is **time-constructible** if $T(n) \geq n$ for every n, and some Turing machine computes the function that maps each $x \in \{0,1\}^*$ to the binary representation of $T(|x|)$ in time $O(T(n))$ (adapted from Arora and Barak 2009, sec. 1.3).

Example 0.3 (Common time-constructible functions)**.** Four common examples, n, $n\lceil \log_2(n+1) \rceil$, n^2, and 2^n, are all time-constructible (Arora and Barak 2009, sec. 1.3). A machine that is allowed $O(T(n))$ steps on inputs of length n can, when T is time-constructible, first write $T(n)$ on a work tape and then use it as a clock that stops a computation after $T(n)$ of its steps. The functions that bound running times in these notes are all of this kind. Time-constructibility rules out functions such as $\lceil \log_2(n+1) \rceil$, which is smaller than n once $n \geq 3$, so a machine cannot even read its whole input in that many steps.

Remark 0.2 (Exact and asymptotic time bounds)*.* Arora and Barak (2009, sec. 1.3) ask the machine for a time-constructible function to finish within $T(n)$ steps, not $O(T(n))$ steps. No machine meets that bound for $T(n) = n$ on the empty input. There $T(0) = 0$, but the machine must take at least one step to write its output. The $O(\cdot)$ version allows for those few extra steps, and the time complexity class uses $O(\cdot)$ for the same reason. With a bound of $cT(n)$ steps for every n, as Arora and Barak (2009, sec. 1.6) write it, the class DTIME(n) would be empty. The two versions agree whenever $T(n) > 0$ for every n.

Theorem 0.1 (The details of the model change running time only polynomially)**.** *Let f be a function from $\{0,1\}^*$ to $\{0,1\}^*$, and let T be time-constructible. Suppose a Turing machine M with k tapes and an alphabet of s symbols computes f in time $T(n)$. Then:*

1. *A Turing machine with k tapes and the alphabet $\{0,1,\square,\triangleright\}$ computes f in time $O(T(n)\log_2 s)$.*
2. *A single-tape machine computes f in time $O(T(n)^2)$.*

A single-tape machine *is the same as a Turing machine, except that it has one tape, which starts with $\triangleright$ followed by the input, which it can read and write, and which holds the output when it halts. The constant in part 2 depends on k, and neither constant depends on f, T, or n (Arora and Barak 2009, sec. 1.3).*

Remark 0.3 (Simulations behind Theorem 0.1). Arora and Barak (2009, sec. 1.3) prove both parts by simulation. For part 1, the new machine writes each symbol of M's alphabet as a block of about $\log_2 s$ binary digits, and it carries out one step of M by reading and rewriting one block on each tape. For part 2, the single tape holds the contents of all k tapes interleaved, with a mark at each head's position. To carry out one step of M, the machine sweeps along the used part of the tape to read the marked symbols, and sweeps back to update them. After t steps, M has used at most $\max(n, t) + 1$ cells of each tape, which is $O(T(n))$ because $T(n) \geq n$, so each sweep takes $O(T(n))$ steps, and the $T(n)$ steps of M take $O(T(n)^2)$ in all.

Remark 0.4 (The Church-Turing thesis). Every model of computation that has been proposed and that can be built, from programming languages to random-access machines, can be simulated by a Turing machine. The *Church-Turing thesis* claims that every function a physical device can compute, a Turing machine can compute. It is a claim about the physical world, not a theorem. A stronger version, which quantum computers may refute, says the simulation costs at most a polynomial increase in running time (Arora and Barak 2009, sec. 1.6). Under that version, the class P is the same whichever reasonable model it is defined with, as Theorem 0.1 shows for the variations of the Turing machine itself.

The universal machine

Definition 0.4 (Machines as strings)**.** A Turing machine is a finite object: its states, its alphabet, and the table of its transition function. So it can be written as a binary string, its **description**. Fix one such encoding, which a machine can decode in a number of steps linear in the length of the description, with two conventions (Arora and Barak 2009, sec. 1.4):

- Every binary string w describes some Turing machine, written M_w. A string that is not a valid encoding describes one fixed machine.
- Every Turing machine has infinitely many descriptions, because the encoding allows padding: if w describes M, so do longer strings that extend w with symbols the decoder ignores.

Example 0.4 (A program file is a description)**.** A Python script saved to disk is a sequence of bytes, so it is a binary string, and the Python interpreter reads that string as a program, much as Definition 0.4 reads a string as a Turing machine. Appending a comment line to the script gives a longer string that describes the same program, which is the padding convention of Definition 0.4. A file that is not valid Python makes the

interpreter stop with a syntax error, which plays the role of the fixed machine that invalid strings describe, for instance one that halts at once.

Definition 0.5 (Pair encoding)**.** Let w and x be binary strings. Their **pair encoding** $\langle w, x \rangle$ is the binary string made of each digit of w written twice, then 01, then x. A machine can recover w and x from $\langle w, x \rangle$ by reading two digits at a time until it reads 01. Arora and Barak (2009, sec. 0.1) use a variant that also doubles the digits of x.

Example 0.5 (Encoding the pair 10 and 0)**.** The pair encoding of $w = 10$ and $x = 0$ is

$$\langle 10, 0 \rangle = 1100010.$$

It consists of 11 and 00 for the two digits of w, then 01, then the single digit of x. Reading two digits at a time gives 11, 00, and then 01, which marks the end of w.

Theorem 0.2 (A universal Turing machine)**.** *There is a Turing machine U that runs any other machine from its description. Its input is the pair encoding $\langle w, x \rangle$ of binary strings w and x. U outputs $M_w(x)$ whenever M_w halts on x, and U runs forever when M_w does not. If M_w halts on x within $T \geq 1$ steps, then U halts within $C\big(|w| + T \log_2(T+1)\big)$ steps. The constant C depends only on the machine M_w. It depends neither on x nor on the padding in w (adapted from Arora and Barak 2009, sec. 1.4).*

Remark 0.5 (Interpreters are universal machines). A Python interpreter is a universal machine in the sense of Theorem 0.2: one fixed program that runs any other program it is given as input. Arora and Barak (2009, sec. 1.4) state Theorem 0.2 with the bound $C\,T \log_2 T$. The $|w|$ term added here counts the steps U spends reading past w, which can be long because of padding, and $T + 1$ keeps the bound positive when $T = 1$. They prove a version with $C\,T^2$ in place of $C\,T \log_2 T$ by an argument like that of Remark 0.3, and they prove the stronger bound in section 1.7. The Kolmogorov complexity of a string is defined with a universal machine that, unlike the one of Theorem 0.2, reads a single program, and halts only on programs none of which is a prefix of another.

References

Arora, Sanjeev, and Boaz Barak. 2009. *Computational Complexity: A Modern Approach*. Cambridge University Press. https://doi.org/10.1017/CBO9780511804090.